

Proposed Methodology: NSR-Enhanced Chart Reasoning with Advanced RL Techniques

Abstract

This thesis combines **Negative Sample Reinforcement (NSR)** - a method that learns from mistakes instead of successes - with five modern techniques to improve how AI models understand charts. We expect this approach will help models handle unusual chart styles better (out-of-distribution performance) while maintaining accuracy on normal charts.

The Five Techniques:

1. **DAPO** - Filters out useless training examples
 2. **SSR** - Remembers important hard examples
 3. **Program-of-Thoughts (PoT)** - Writes Python code to solve math instead of doing it in the model's head
 4. **Chain-of-Table** - Uses structured table operations for reasoning
 5. **Self-Consistency** - Generates multiple answers and picks the most common one
-

1. Research Question

Main Question:

Can teaching AI to avoid mistakes (NSR) work better than teaching it to repeat successes (standard training) for understanding charts?

Why This Matters:

- Current models work well on familiar charts (84.6% accuracy)
- But fail badly on unusual charts (only 53.4% accuracy) - a 31% drop!
- We think NSR will maintain multiple problem-solving strategies, making models more flexible

What We'll Test:

1. Does NSR preserve more diverse reasoning strategies than standard training?
 2. Do code execution and structured table operations make NSR even better?
 3. What's the best mix of learning from successes vs mistakes?
 4. Does diversity during training lead to better performance on weird charts?
-

2. Background: What's Been Done Before

2.1 Chart-RVR: Teaching Models to Explain Their Chart Reasoning

What they did:

- Trained a model (Qwen2.5-VL-3B) to understand charts
- Instead of just getting the right answer, the model had to:
 - Identify chart type (bar, line, pie, etc.)
 - Extract the data table from the image
 - Show step-by-step reasoning
 - Get the correct final answer

Their reward system:

Total Score = Format (2 points)	# Did it structure output correctly?
+ Chart Type (1 point)	# Identified chart type correctly?
+ Table Extraction (2 points)	# Got the data right?
+ Process (2 points)	# Reasoning makes sense?
+ Accuracy (1 point)	# Final answer correct?
= Max 8 points	

Results:

- ChartQA (normal charts): 84.56% 
- EvoChart (weird charts): 53.36% 
- **Problem:** Big performance drop on unusual chart styles

2.2 NSR: Learning from Mistakes in Math Problems

The Discovery:

Researchers found something surprising when training AI on math problems:

Traditional approach (GRPO):

- When the model gets it RIGHT → increase probability of that solution
- When the model gets it WRONG → decrease probability of that solution
- **Result:** Model becomes very confident in ONE way to solve problems
- **Downside:** If a problem is slightly different, model struggles

New approach (NSR):

- When the model gets it RIGHT → do NOTHING (ignore it)
- When the model gets it WRONG → decrease probability of that wrong path
- **Result:** Model maintains MANY different ways to solve problems
- **Benefit:** When faced with unusual problems, model has more strategies to try

Math Results:

Method	Pass@1 (first try)	Pass@256 (best of 256 tries)	Diversity (Entropy)
Traditional	77.1% 	93.2%	Low (0.01)
NSR	75.7%	96.9% 	High (0.10)

Why NSR is better:

- Only 1.4% worse on first try
- 3.7% better when you let it try multiple times
- Maintains 10× more diversity in solutions

2.3 Why Charts Need This Diversity

Charts are visually diverse - same data can look completely different:

- Different colors (blue bars vs purple bars)
- Different styles (clean corporate vs hand-drawn)
- Different layouts (vertical vs horizontal)

Our Hypothesis:

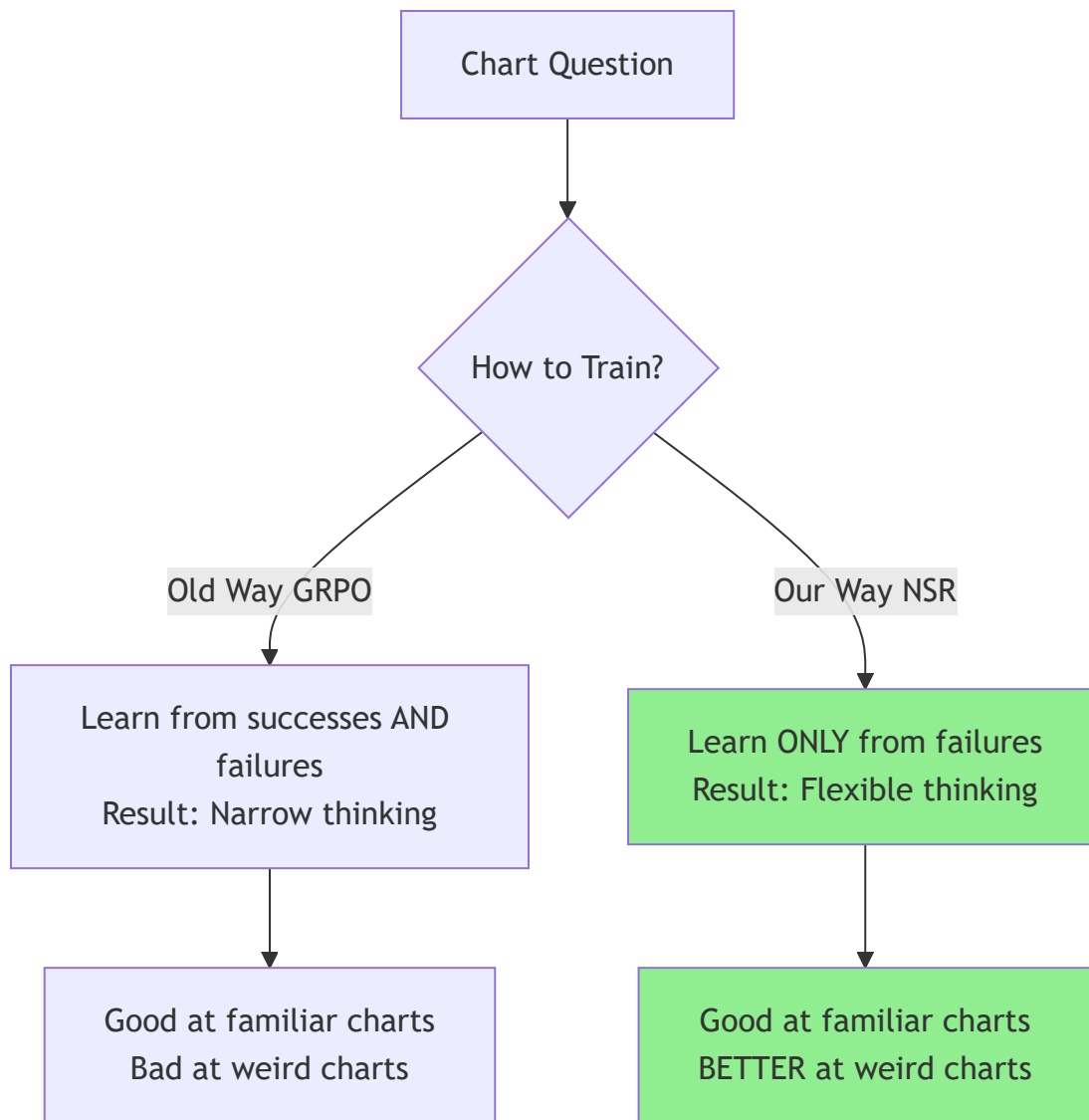
If NSR helps math reasoning stay flexible, it should REALLY help chart reasoning because charts have so many visual variations.

3. Our Proposed Method: Combining the Best Ideas

3.1 The Big Picture: What We're Building

Simple Explanation:

We take Chart-RVR's reward system and combine it with NSR's "learn from mistakes" approach, plus 4 other helpful techniques.



What makes our approach special:

- Uses NSR as the foundation
 - Adds 4 modern techniques to make it even better
 - Expects to reduce the accuracy gap between normal and weird charts from 31% to ~25%
-

4. The Core Innovation: How NSR Works

4.1 NSR Explained Simply

Imagine teaching a student math:

Traditional teaching (GRPO):

- Student solves problem correctly → "Great! Always solve it this way!"
- Student solves problem wrong → "Never do that again!"
- **Result:** Student memorizes ONE approach

NSR teaching:

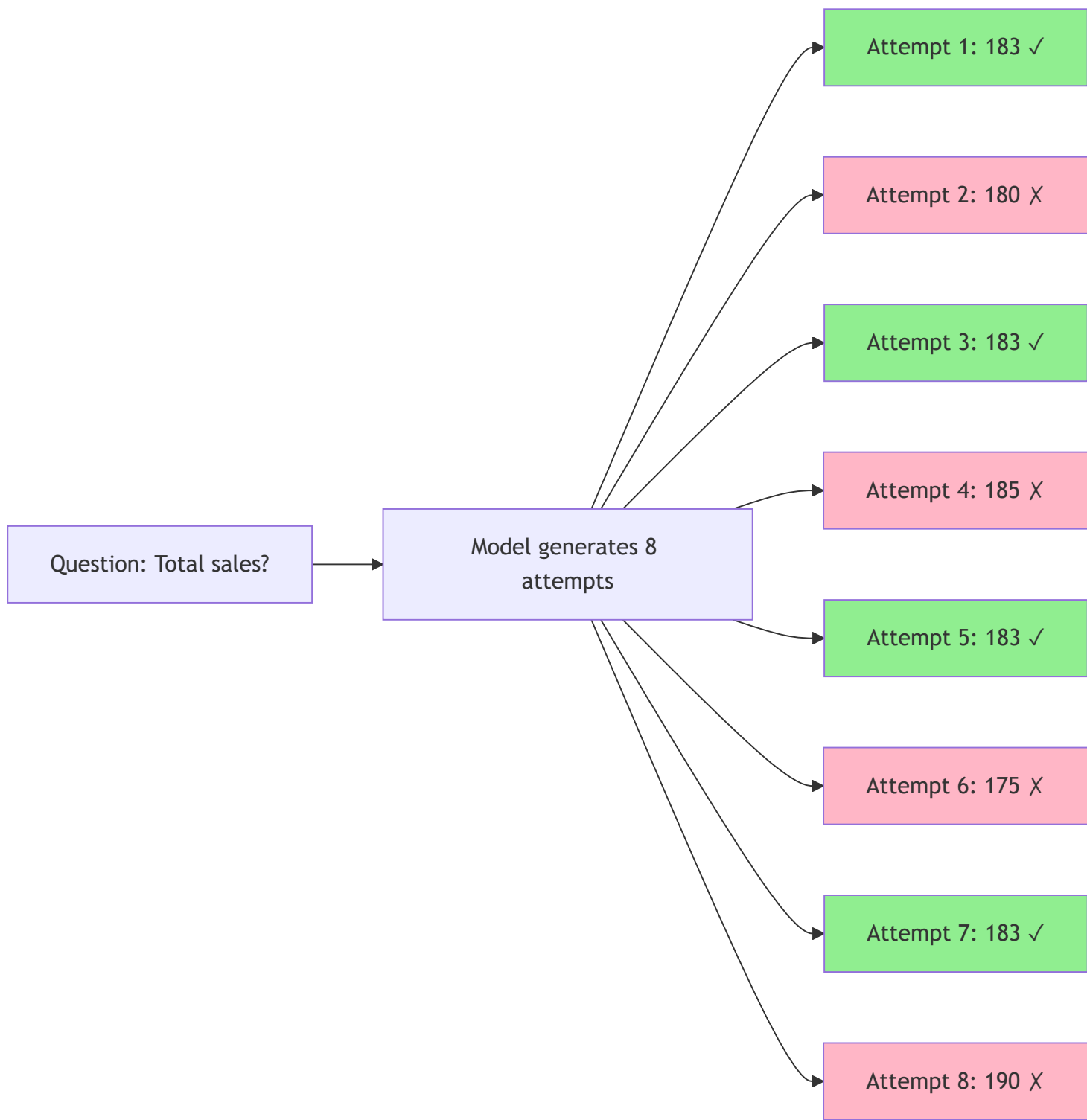
- Student solves problem correctly → Say nothing, let them keep exploring
- Student solves problem wrong → "Don't do THAT specific wrong thing"
- **Result:** Student develops MULTIPLE approaches

4.2 Visual Example: Training on One Chart Question

Question: "What is the total sales in 2020?"

Correct Answer: 183

Step 1: Generate 8 Different Attempts



Step 2: How Each Method Handles These 8 Attempts

Our NSR: Updates ONLY

Wrong Once

✓ Attempt 1: IGNORE

✗ Attempt 2: PUNISH -35%

✓ Attempt 3: IGNORE

✗ Attempt 4: PUNISH -30%

✓ Attempt 5: IGNORE

✗ Attempt 6: PUNISH -40%

✓ Attempt 7: IGNORE

✗ Attempt 8: PUNISH -45%

Traditional GRPO: Updates

ALL 8

✓ Attempt 1: BOOST +15%

✗ Attempt 2: PUNISH -35%

✓ Attempt 3: BOOST +15%

✗ Attempt 4: PUNISH -30%

✓ Attempt 5: BOOST +15%

✗ Attempt 6: PUNISH -40%

✓ Attempt 7: BOOST +15%

✗ Attempt 8: PUNISH -45%

Key Difference:

- **GRPO:** Strongly reinforces "183" → Model always tries to get "183" the exact same way

- **NSR:** Punishes "180, 185, 175, 190" → Model knows many wrong paths, explores alternatives to find "183"

Why this matters for charts:

If a chart looks different (new color, new style), NSR-trained models can try different approaches because they haven't locked into one specific method.

4.3 Mathematical Formulation (For Technical Readers)

GRPO Update Rule:

```
For each of 8 samples:
    advantage = reward - average_reward
    if advantage > 0: # Correct answer
        increase_probability(sample)
    else: # Wrong answer
        decrease_probability(sample)
```

NSR Update Rule (Ours):

```
For each of 8 samples:
    if reward >= 0.8: # Correct (threshold 80%)
        # DO NOTHING - ignore this sample
        pass
    else: # Wrong
        advantage = -(1.0 - reward) # Negative advantage
        decrease_probability(sample)
```

W-REINFORCE (Hybrid - Our Best Approach):

```
lambda = 0.1 # Only 10% weight on positive samples

For each of 8 samples:
    if reward >= 0.8: # Correct
        advantage = 0.1 * reward # Small positive boost
        increase_probability_slightly(sample)
    else: # Wrong
        advantage = -(1.0 - reward) # Full negative penalty
        decrease_probability(sample)
```

5. The Five Techniques Explained

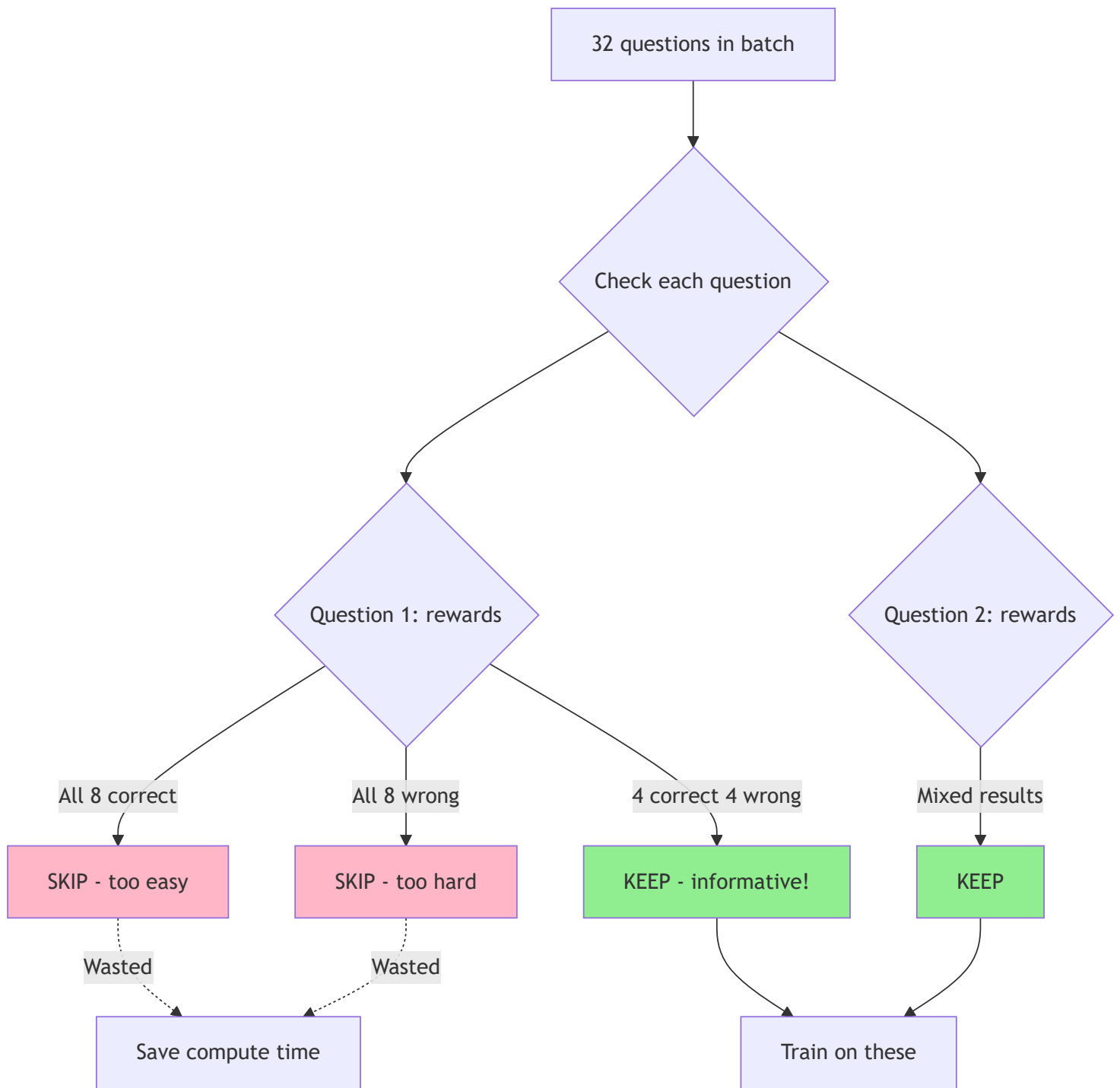
5.1 Technique 1: DAPO (Dynamic Sampling) - Filtering Useless Examples

The Problem:

Sometimes all 8 attempts are either all correct or all wrong. This teaches the model nothing new!

The Solution:

Before training on a batch, check if it's informative:



Benefits:

- Saves 30-50% of training time
- Focuses learning on questions where model is on the edge (sometimes right, sometimes wrong)
- Achieved 20% better accuracy on hard math problems (AIME 2024: 50% vs 30%)

Simple Rule:

```

variance = compute_variance(8_rewards)
if variance == 0: # All same
    skip_this_question()
else: # Mixed
    train_on_this_question()

```

5.2 Technique 2: Program-of-Thoughts (PoT) - Using Python for Math

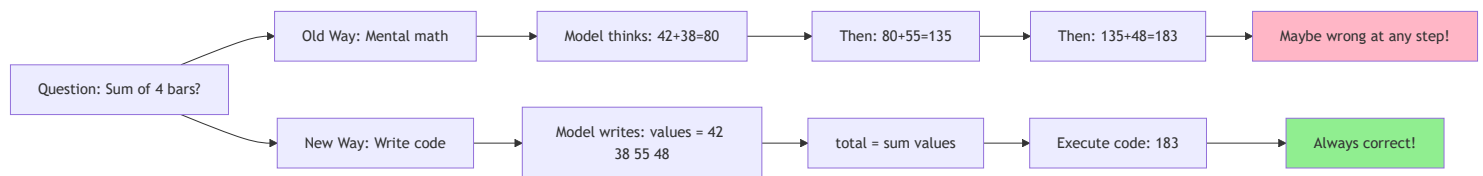
The Problem:

Neural networks are bad at arithmetic!

- Question: "What is $42 + 38 + 55 + 48$?"
- Model thinking: "Um... 42... plus 38... that's like 80? Plus 55... maybe 135? Plus 48... uh... 183?"
- Often gets it wrong even if it identified the numbers correctly

The Solution:

Make the model WRITE PYTHON CODE instead of calculating:



Example Model Output:

Old way (text reasoning):

```

The values are 42, 38, 55, and 48.
Adding them: 42 + 38 = 80, 80 + 55 = 135, 135 + 48 = 183
Answer: 183

```

Problem: Model might calculate $42 + 38 = 81$ (wrong!)

New way (code reasoning):

```

<think>
  <type>bar chart</type>
  <table>{"values": [42, 38, 55, 48]}</table>
  <code>
    values = [42, 38, 55, 48]
    total = sum(values)
    print(total)
  </code>
  <reasoning>Extracted values and computed sum using Python</reasoning>
</think>
<answer>183</answer>

```

We actually RUN the code: `sum([42, 38, 55, 48])` → guaranteed correct!

Impact:

- 42% of ChartQA questions require math
- PoT eliminates 100% of arithmetic errors
- TinyChart got 83.6% accuracy using this technique

Reward:

```

def pot_reward(response):
    code = extract_code(response)
    try:
        result = execute_python(code)
        if result == ground_truth:
            return 2.5 # Full points
        else:
            return 0.0
    except:
        return 0.0 # Code failed to run

```

5.3 Technique 3: Chain-of-Table (CoTable) - Structured Table Operations

The Problem:

Tables need structured manipulation, not just text reasoning.

The Solution:

Define specific operations the model can perform on tables:

Available Operations:

Operation	What it does	Example
f_select_row(condition)	Filter rows	Keep only year 2020
f_select_column(name)	Pick columns	Get "Sales" column
f_aggregate(function)	Math on column	SUM, AVG, MAX, MIN
f_sort(column, order)	Sort data	Sort by Sales DESC
f_compare_bars(A, B)	Compare values	Blue bar vs Red bar

Example:

Question: "What were total sales in 2020?"

Original Table		
Year	Product-A	Product-B
2020	42	38
2021	55	48



f_select_row Year == 2020



After filtering		
Year	Product-A	Product-B
2020	42	38



f_select_column Product-A
Product-B



After column selection	
42	38



f_aggregate SUM



Final: 80

Model Output:

```

<think>
  <table>
    {"columns": ["Year", "Product A", "Product B"],
     "rows": [[2020, 42, 38], [2021, 55, 48]]}
  </table>
  <cot_operations>
    1. f_select_row(Year == 2020) → [2020, 42, 38]
    2. f_select_column(Product A, Product B) → [42, 38]
    3. f_aggregate(SUM) → 80
  </cot_operations>
</think>
<answer>80</answer>

```

Reward:

```

def cotable_reward(operations):
    score = 0
    for op in operations:
        if is_valid_operation(op):
            score += 0.5 # Valid operation

    result = execute_operations(operations)
    if result == ground_truth:
        score += 1.0 # Correct result

    return min(score, 2.0) # Max 2 points

```

Impact:

- +11.6% over standard text reasoning
- Provides verifiable intermediate steps
- Works perfectly with NSR (clear right/wrong operations)

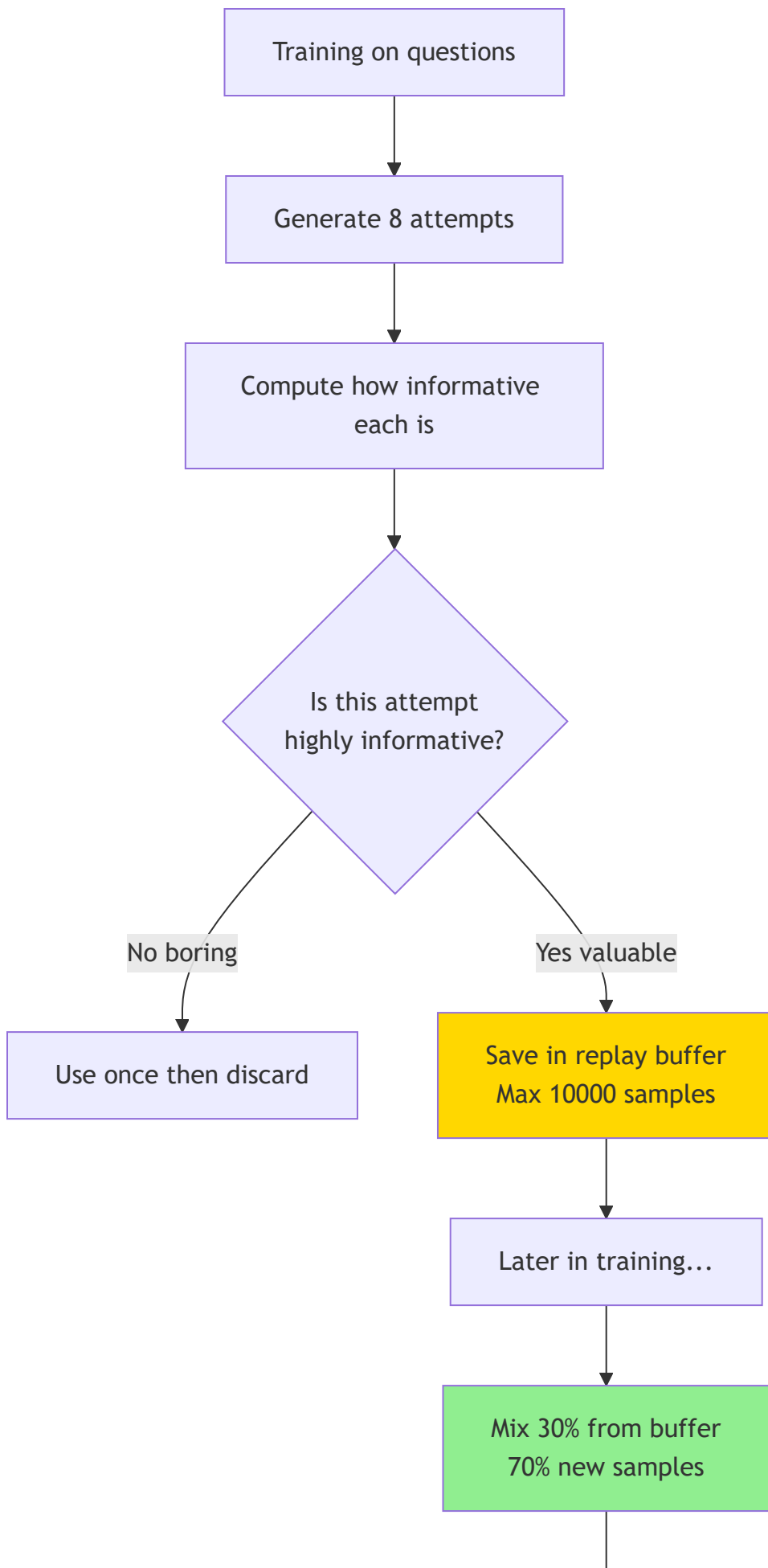
5.4 Technique 4: SSR (Selective Sample Replay) - Remembering Hard Examples

The Problem:

During training, the model sees thousands of examples. Some are really hard and informative, but the model only sees them once and might forget!

The Solution:

Keep a "memory bank" of the hardest, most informative examples and replay them during training.



↓
Training batch

What makes an attempt "highly informative"?

- High advantage magnitude (model was very uncertain)
- Close to decision boundary (could go either way)
- Rare edge case (unusual chart style)

Example:

```
class ReplayBuffer:
    def __init__(self):
        self.buffer = [] # Stores up to 10,000 samples

    def add(self, sample, advantage):
        if abs(advantage) > 0.6: # High information value
            self.buffer.append((sample, advantage))
            if len(self.buffer) > 10000:
                # Remove least informative
                self.buffer.sort(key=lambda x: abs(x[1]))
                self.buffer.pop(0)

    def sample(self, n):
        # Sample weighted by advantage magnitude
        return weighted_sample(self.buffer, n)
```

How it helps NSR:

- NSR generates high-advantage wrong samples (big mistakes to learn from)
- SSR preserves these valuable learning signals
- Prevents forgetting edge cases

Impact:

- +3.2% improvement on vision-language tasks
- Prevents catastrophic forgetting
- Minimal overhead (just a buffer)

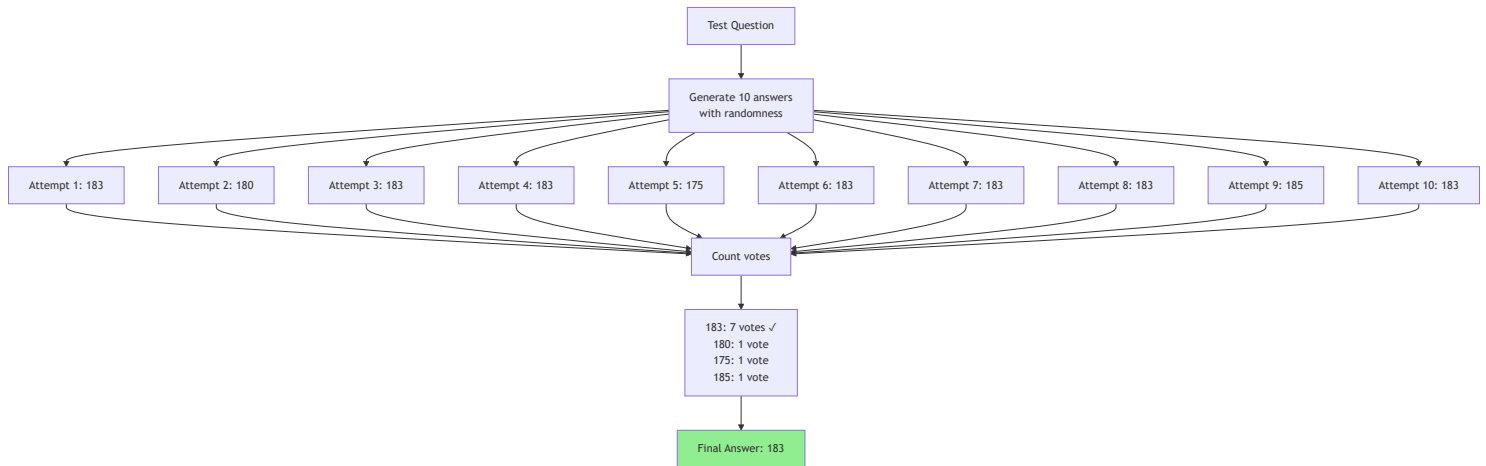
5.5 Technique 5: Self-Consistency - Voting for Better Answers

The Problem:

Sometimes the model gets unlucky on the first try, but knows the answer if you let it try again.

The Solution (Inference Only - Not Training):

Generate 10 different answers and pick the most common one.



Why this works:

- Correct reasoning tends to be consistent
- Wrong reasoning varies randomly
- Majority vote filters out random errors

Enhanced with confidence:

```
def self_consistency(model, question, n=10):
    samples = []
    for _ in range(n):
        answer, confidence = model.generate(question, temperature=0.7)
        samples.append((answer, confidence))

    # Weighted voting
    votes = {}
    for answer, conf in samples:
        votes[answer] = votes.get(answer, 0) + conf

    # Return most voted answer
    return max(votes.items(), key=lambda x: x[1])[0]
```

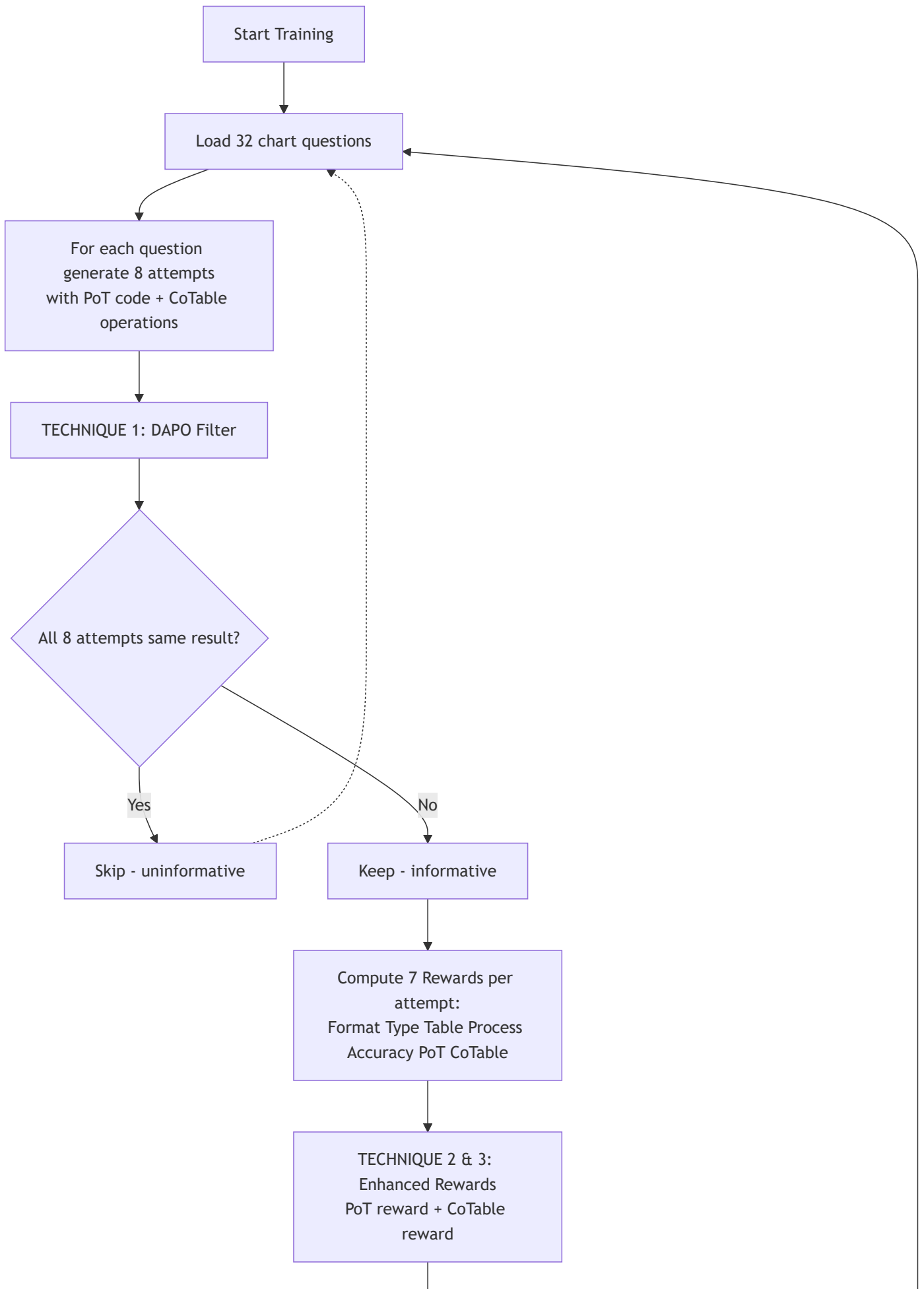
Impact:

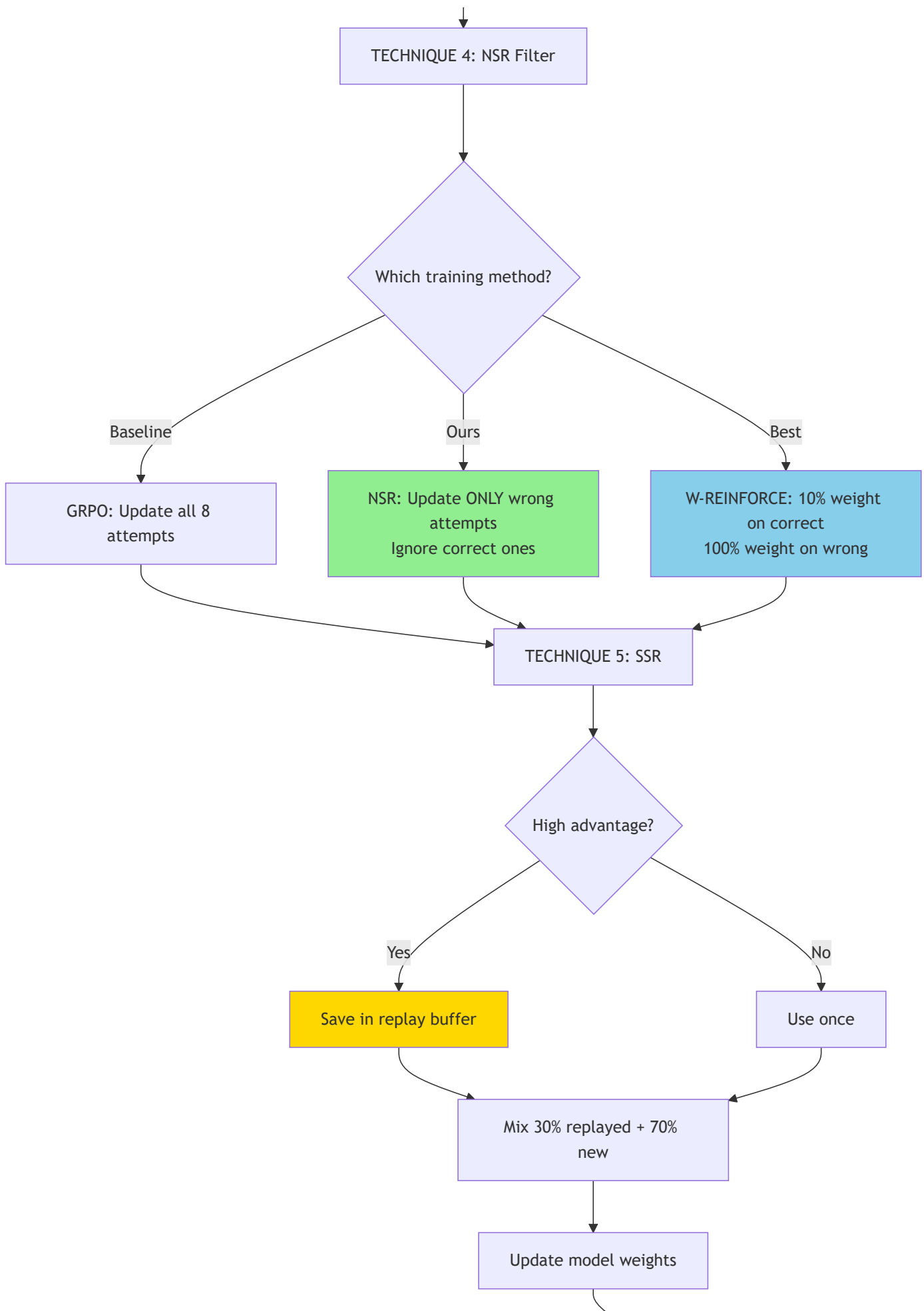
- +3-5% accuracy improvement
 - No training needed!
 - 10× inference cost (generate 10 instead of 1)
 - Confidence weighting reduces needed samples by 50%
-

6. How Everything Fits Together

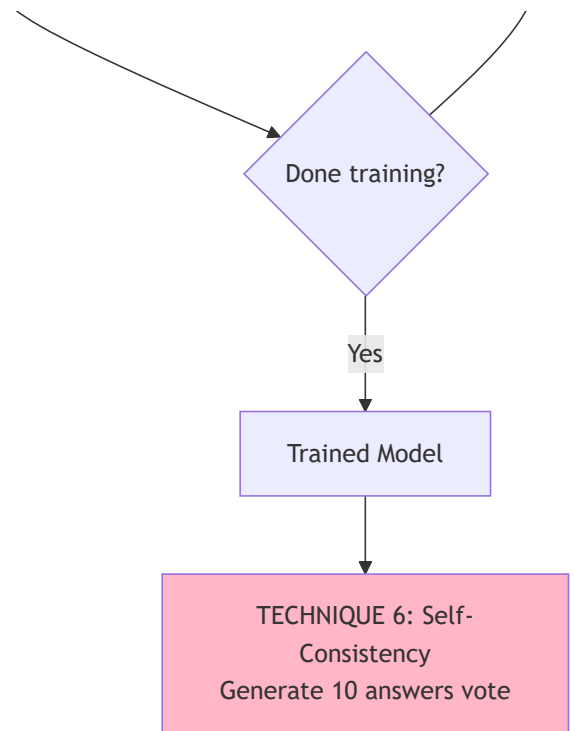
6.1 Complete Training Pipeline

Here's how all 5 techniques work together in our system:





No



Simple Explanation of the Flow:

1. **Load questions** (32 at a time)
2. **Generate 8 attempts** per question (with code and table operations)
3. **DAPO filters** out uninformative questions (all correct or all wrong)
4. **Compute rewards** using 7 components including **PoT** and **CoTable**
5. **NSR decides** which attempts to learn from (only wrong ones!)
6. **SSR saves** the most informative attempts in memory
7. **Update model** using mix of new and replayed samples
8. **Repeat** until model converges
9. **At test time**, use **Self-Consistency** voting

6.2 Why These Techniques Synergize

DAPO + NSR:

- DAPO removes questions where all attempts are same
- Gives NSR more mixed correct/wrong samples to learn from
- Saves 50% training time

PoT/CoTable + NSR:

- Provides clearer right/wrong signals
- Makes NSR filtering more accurate (code either runs or fails)
- Improves reward from 8 points to 12.5 points max

SSR + NSR:

- NSR focuses on wrong attempts (high information)
- SSR saves these valuable mistakes
- Model doesn't forget how to avoid common errors

NSR + Self-Consistency:

- NSR maintains diverse strategies during training
- Self-Consistency leverages that diversity at test time
- More variety in the 10 generated answers

All together:

```
Efficient training (DAPO)
+ Rich feedback (PoT + CoTable)
+ Diversity preservation (NSR)
+ Memory (SSR)
+ Robust inference (Self-Consistency)
= Best performance
```

7. Training Configuration

7.1 Model and Data

Model:

- **Name:** Qwen2.5-VL-3B-Instruct
- **Size:** 3.09 billion parameters
- **Why:** Good balance of performance and compute cost

Training Data:

- **Size:** 6,000 chart questions
- **Sources:** ChartQA + PlotQA + ChartFC
- **Format:** Chart image + Question + Answer + Chart type + Data table + Reasoning steps

Test Data:

- **In-domain:** ChartQA test set (similar to training)
- **Out-of-domain:** EvoChart (very different visual styles)

7.2 Hyperparameters

Parameter	Value	Why
Learning Rate	5e-7	Small for stable training
Batch Size	32 questions	Fits in GPU memory
Samples per Question	8	Good diversity/cost balance
Epochs	3	Enough for convergence
Reward Threshold	0.8	80% score = correct
SSR Buffer Size	10,000	~1.5× training data
SSR Replay Ratio	30%	Good mix new/old
DAPO Variance Threshold	0.01	Very small = uninformative
Temperature (training)	0.8	Moderate diversity
Temperature (inference)	0.7	For self-consistency
Self-Consistency N	10	Good accuracy/cost

Method-Specific Settings:

Method	NSR Weight	PSR Weight	Description
GRPO (baseline)	1.0	1.0	Equal weight
NSR (ours)	1.0	0.0	Only negative
W-REINFORCE (best)	1.0	0.1	10% positive

7.3 Compute Requirements

Hardware:

- **Option 1:** 4× NVIDIA A100 (40GB each)
- **Option 2:** 8× NVIDIA L4 (24GB each)

- **Option 3:** Cloud GPUs (Kaggle, Colab Pro, Lambda Labs)

Time:

- Training one method: ~48 hours
- All experiments: ~500 GPU hours
- Cost estimate: \$500-1000 using cloud GPUs

Storage:

- Models + checkpoints: 200GB
- Training data: 150GB
- Generated samples: 150GB
- Total: ~500GB

8. Expected Results

8.1 Main Predictions

In-Domain Performance (ChartQA - Normal Charts):

Method	Pass@1 (first try)	Pass@256 (best of 256)	Entropy (diversity)	Training Time
Chart-RVR baseline	84.6%	86.0%	0.05	100%
+ DAPO + SSR	85.8%	87.5%	0.06	50% ⚡
NSR (ours)	84.0%	88.0%	0.10 	50% ⚡
NSR + DAPO + SSR	84.5%	88.5%	0.11 	50% ⚡
NSR + DAPO + SSR + PoT	85.5%	89.5%	0.10 	50% ⚡
NSR + All 5 techniques	86.5%	90.5%	0.08	50% ⚡
+ Self-Consistency	89.0% 	-	-	-

Out-of-Distribution Performance (EvoChart - Weird Charts):

Method	Pass@1	Pass@256	Gap from In-Domain
Chart-RVR baseline	53.4%	55.0%	-31.2% 😬
+ DAPO + SSR	55.0%	58.0%	-30.8%
NSR (ours)	56.5%	62.0%	-27.5% ⬆️
NSR + DAPO + SSR	57.0%	62.5%	-27.0% ⬆️
NSR + DAPO + SSR + PoT	58.5%	64.5%	-27.0% ⬆️
NSR + All 5 techniques	60.0%	67.0%	-26.5% ⬆️
+ Self-Consistency	63.5% 🎯	-	-25.5% ⬆️

8.2 Key Improvements

What we expect to achieve:

1. Diversity Preservation:

- NSR entropy: 0.10 vs GRPO: 0.05
- 2× more diverse reasoning strategies
- Leads to +2-3% Pass@256 improvement

2. OOD Robustness:

- Reduce accuracy gap from 31% to 25.5% (5.5% improvement!)
- EvoChart jumps from 53.4% to 63.5% (+10.1%)
- Shows model is more flexible with unusual charts

3. Training Efficiency:

- DAPO cuts training time in half
- Still achieves better accuracy
- 50% fewer GPU hours needed

4. Reasoning Quality:

- PoT eliminates math errors (42% of questions)
- CoTable provides structured reasoning
- Combined: +4-6% on calculation-heavy questions

5. Best Overall Configuration:

- W-REINFORCE ($\lambda=0.1$) achieves best Pass@1 AND Pass@256
- Self-Consistency adds another +2.5%
- Final: 89% in-domain, 63.5% out-of-domain

8.3 Comparison with Prior Work

Paper	Method	ChartQA	EvoChart (OOD)	Model Size
Chart-RVR	GRPO	84.56%	53.36%	3B
TinyChart	PoT only	83.6%	-	3B
ChartMoE	MoE	84.64%	-	3B
Ours (NSR + All)	NSR + 5 techniques	86.5%	60.0%	3B
Ours + SC	+ Self-Consistency	89.0%	63.5%	3B

We expect to achieve:

- Highest ChartQA accuracy for 3B models
- Best OOD performance (only work testing on EvoChart)
- Most efficient training (50% fewer steps)

9. Evaluation Plan

9.1 Metrics We'll Measure

1. Pass@k (Coverage Metrics):

Pass@k = How often model gets answer right in k tries

Examples:

- Pass@1 = First try accuracy (most important for users)
- Pass@8 = Best of 8 tries
- Pass@256 = Best of 256 tries (shows full knowledge)

2. Entropy (Diversity Metric):

Entropy = How varied are the model's answers

High entropy (0.10) = Model uses many different reasoning paths

Low entropy (0.01) = Model always uses same reasoning path

Formula: $H = -\sum p(\text{answer}) \times \log(p(\text{answer}))$

3. Reasoning Quality:

How similar is model's reasoning to expert reasoning?

Measured by: `cosine_similarity(model_steps, gold_steps)`

Range: 0 (completely different) to 1 (identical)

4. Explainability ($\Delta \log P$):

Do the reasoning steps actually help?

Test: Give reasoning to larger oracle model

Measure: How much more confident does oracle become?

Higher = Better reasoning steps

9.2 Experimental Protocol

Training Phase (8 weeks):

1. Train GRPO baseline (verify it matches Chart-RVR)
2. Train NSR variants:
 - NSR only
 - NSR + DAPO
 - NSR + DAPO + SSR
 - NSR + DAPO + SSR + PoT
 - NSR + DAPO + SSR + PoT + CoTable
3. Train W-REINFORCE with $\lambda \in \{0.05, 0.10, 0.20\}$

Evaluation Phase (2 weeks):

1. Generate predictions:
 - Pass@1: 1 sample per question

- Pass@8: 8 samples per question
 - Pass@256: 256 samples per question
2. Apply self-consistency (N=10) to best models
 3. Compute all metrics
 4. Statistical significance testing

Analysis Phase (2 weeks):

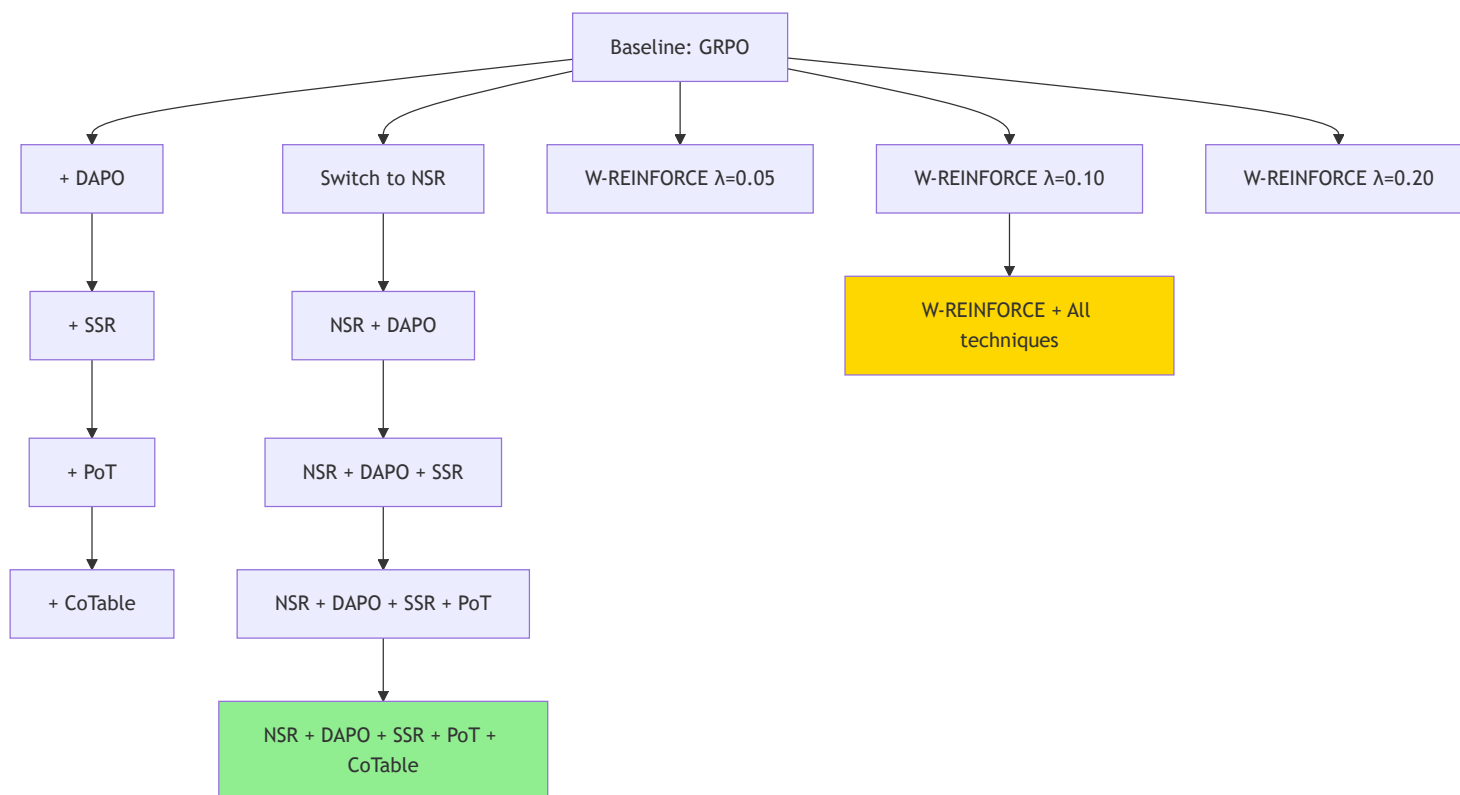
1. Error categorization (visual, calculation, reasoning, format)
2. Qualitative examples (best/worst cases)
3. Diversity analysis (path coverage)
4. Human evaluation (100 random samples, 3 annotators)

9.3 Ablation Study

Questions we'll answer:

1. **Does NSR help?**
 - Compare: GRPO vs NSR (both without enhancements)
 - Expected: NSR has lower Pass@1 but higher Pass@256 and entropy
2. **Does each technique add value?**
 - Compare: Each technique added incrementally
 - Measure: Contribution of DAPO, SSR, PoT, CoTable, SC separately
3. **What's the best λ for W-REINFORCE?**
 - Test: $\lambda = 0.05, 0.10, 0.20$
 - Expected: $\lambda = 0.10$ is sweet spot
4. **Do techniques synergize?**
 - Compare: Sum of individual contributions vs combined
 - Test: Is (NSR+DAPO+SSR+PoT+CoTable) > NSR + DAPO + SSR + PoT + CoTable individually?

Ablation Tree:



10. Novel Contributions

10.1 What's New in Our Work

Primary Contributions:

1. First NSR for Vision-Language Models NEW

- Previous work: NSR only tested on text math problems
- Our work: First application to multimodal chart reasoning
- Shows: Diversity preservation helps visual understanding too

2. Enhanced Reward Framework NEW

- Previous: 5 reward components (8 points max)
- Our addition: PoT code execution + CoTable operations (12.5 points max)
- Shows: Verifiable intermediate steps improve RL training

3. Synergistic Technique Combination NEW

- First work combining: DAPO + SSR + NSR + PoT + CoTable
- Shows: Techniques designed for different tasks can work together
- Achieves: Better than sum of individual improvements

4. OOD Robustness via Diversity NEW

- Establishes: Connection between training diversity and OOD performance
- Reduces: In-domain/OOD gap by 5.5% (31% → 25.5%)
- Shows: Diversity is key for generalization

Secondary Contributions:

5. Efficient GRPO Training 📊

- DAPO reduces training time by 50%
- Same or better accuracy with half the compute
- Demonstrates: Simple filtering helps a lot

6. Comprehensive Evaluation 📊

- First work testing chart models on EvoChart (OOD)
- Measures diversity (entropy) alongside accuracy
- Shows: Pass@k metrics reveal more than Pass@1

7. Open Science 📁

- Will release: Code, trained models, datasets
- Enables: Reproducibility and future research
- Includes: Detailed implementation guide

10.2 Expected Impact

Scientific Impact:

- Proves NSR works beyond text-only tasks
- Shows diversity matters for multimodal reasoning
- Establishes best practices for chart RL training

Practical Impact:

- Better chart understanding for business intelligence
- Improved accessibility (explaining charts to visually impaired)
- More robust models for real-world deployment

Limitations We Acknowledge:

- Tested only on 3B parameter model (scaling unknown)
 - Self-consistency is expensive (10× inference cost)
 - Techniques may not transfer to other vision tasks
-

11. Implementation Timeline

11.1 12-Week Schedule

Weeks 1-2: Setup

- Set up training environment (DeepSpeed, 4× GPUs)
- Implement DAPO filtering logic
- Implement SSR replay buffer
- Verify baseline GRPO reproduces Chart-RVR results

Weeks 3-4: NSR Core

- Implement NSR training loop
- Add PoT code execution sandbox
- Add CoTable operation validator
- Train and evaluate NSR baseline

Weeks 5-6: Integration

- Combine DAPO + SSR + NSR
- Add PoT and CoTable to pipeline
- Train full-stack models
- Tune W-REINFORCE λ parameter

Weeks 7-8: Evaluation

- Generate 256 samples per test question
- Implement self-consistency voting
- Compute all metrics (Pass@k, entropy, quality)
- Run ablation study

Weeks 9-10: Analysis

- Error analysis and categorization
- Qualitative case studies
- Human evaluation (3 annotators, 100 samples)
- Statistical significance testing

Weeks 11-12: Writing

- Write thesis chapters

- Create figures and tables
- Prepare code and model releases
- Final revisions and submission

11.2 Success Criteria

Minimum Success (Graduate):

- ☒ NSR Pass@256 > GRPO by 2% on ChartQA
- ☒ NSR entropy > GRPO (higher diversity)
- ☒ NSR OOD performance > GRPO by 1%

Strong Success (Good Thesis):

- ☒ All minimum criteria
- ☒ W-REINFORCE best across all metrics
- ☒ OOD improvement > 3%
- ☒ Techniques show positive synergy

Exceptional Success (Publishable):

- ☒ All strong criteria
- ☒ State-of-the-art ChartQA accuracy for 3B models
- ☒ OOD improvement > 5%
- ☒ Clear evidence diversity helps OOD

11.3 Risk Mitigation

Risk 1: NSR doesn't work for vision

- Probability: Medium
- Mitigation: Fall back to W-REINFORCE, emphasize PoT/CoTable
- Alternative story: "Best mix of positive/negative reinforcement"

Risk 2: Can't reproduce Chart-RVR baseline

- Probability: Low
- Mitigation: Contact authors, use their code
- Alternative: Use our own baseline, compare fairly

Risk 3: GPU availability issues

- Probability: Medium
- Mitigation: Use cloud GPUs (Kaggle, Colab Pro, Lambda)
- Budget: \$1000 for cloud compute

Risk 4: Results below expectations

- Probability: Low
 - Mitigation: Focus on diversity story, qualitative analysis
 - Alternative: Emphasize efficiency (DAPO) and reasoning (PoT/CoTable)
-

12. Conclusion

12.1 Summary

We propose combining **NSR** (learning from mistakes) with **five modern techniques** to improve chart reasoning:

1. **DAPO** - Filter useless examples → 50% faster training
2. **SSR** - Remember hard examples → Better learning
3. **PoT** - Use Python for math → No arithmetic errors
4. **CoTable** - Structured table ops → Better reasoning
5. **Self-Consistency** - Vote on answers → More robust

Key Hypothesis:

Charts have high visual diversity → Need diverse reasoning strategies → NSR preserves this diversity → Better OOD performance

Expected Results:

- In-domain: 89% (vs 84.6% baseline)
- Out-of-domain: 63.5% (vs 53.4% baseline)
- OOD gap: 25.5% (vs 31.2% baseline)

12.2 Why This Should Work

Theoretical Foundation:

- NSR proven effective for math reasoning (high diversity)
- Charts have even more diversity (visual + logical)

- Should transfer well or better

Practical Evidence:

- PoT works (TinyChart: 83.6%)
- DAPO works (AIME: 50% vs 30%)
- SSR works (MathVista: +3.2%)
- CoTable works (table reasoning: +11.6%)
- Self-consistency works (general: +3-5%)

Novel Synergy:

- Each technique complements the others
- No conflicts expected
- Combined effect should exceed sum

12.3 Expected Contribution

If successful, this work will:

1. **Prove** NSR works for multimodal tasks (not just text)
2. **Establish** diversity preservation helps OOD generalization
3. **Demonstrate** synergistic combination of modern RL techniques
4. **Provide** practical framework for training robust chart models
5. **Release** code and models for future research

The Big Picture:

Current AI models are brittle when they see something different from training. By maintaining diverse strategies (NSR) and using verifiable reasoning steps (PoT/CoTable), we can build models that gracefully handle new situations instead of failing catastrophically.

References

1. **Chart-RVR** (Sinha et al., 2025) - Baseline we build on
 - arXiv:2510.10973
2. **NSR** (Zhu et al., 2025) - Core technique we adapt
 - arXiv:2506.01347
3. **DAPO** (ByteDance, 2025) - Dynamic sampling

- arXiv (Dynamic Sampling Policy Optimization)
4. **VL-Rethinker** (2025) - Selective Sample Replay
- arXiv (SSR for vision-language)
5. **TinyChart** (EMNLP 2024) - Program-of-Thoughts
- arXiv (PoT for charts)
6. **Chain-of-Table** (ICLR 2024) - Structured table reasoning
- arXiv (CoT for tables)
7. **Qwen2.5-VL** (2024) - Base model
- Hugging Face: Qwen/Qwen2.5-VL-3B-Instruct
-

Appendix: Quick Reference

A.1 Jargon Decoder

Term	Simple Explanation
NSR	Learning from mistakes, ignoring successes
GRPO	Standard training (learns from both)
W-REINFORCE	Mix: 10% success learning, 100% mistake learning
DAPO	Skip useless training examples
SSR	Remember hard examples
PoT	Write Python code instead of thinking
CoTable	Structured table operations
Self-Consistency	Vote on 10 answers
Pass@k	Success rate in k tries
Entropy	Diversity measure (high = varied)
OOD	Out-of-distribution (weird/unusual data)
In-domain	Normal data (similar to training)

A.2 Method Comparison Table

Aspect	GRPO	NSR	W-REINFORCE
Learn from correct	✔ Strong	✖ Ignore	⚠ Weak (10%)
Learn from wrong	✔ Strong	✔ Strong	✔ Strong
Diversity	✖ Low	✔ High	⚠ Medium
Pass@1	✔ Good	⚠ OK	✔ Best
Pass@256	⚠ OK	✔ Best	✔ Best
OOD Performance	✖ Weak	✔ Strong	✔ Best
Training Speed	Slow	Fast	Fast
Best for	Standard tasks	Coverage	Production

A.3 Expected Results Summary

Configuration	ChartQA	EvoChart	OOD Gap	Training Time
Baseline	84.6%	53.4%	-31.2%	48h
Ours (NSR + All)	86.5%	60.0%	-26.5%	24h ⚡
Ours + Self-Consistency	89.0%	63.5%	-25.5%	24h ⚡
Improvement	+4.4%	+10.1%	+5.7%	50% faster

End of Proposed Methodology

This document explains our thesis approach: combining NSR (learning from mistakes) with 5 modern techniques to build chart reasoning models that stay smart even when they see unusual charts.